EWB COURSES - AWS INTERNSHIP

AWS VPC Peering Project Report

Private communication between two Amazon VPCs using VPC peering, a bastion host, and private EC2 instances - proved by a failed ping before peering, successful pings in both directions after peering, SSH through the bastion, and a controlled security-group failure and recovery.

Student: Rahul Kumar

Internship: AWS Internship (EWB Courses)

AWS Region: Asia Pacific (Sydney) - ap-southeast-2

AWS Account ID: `815756779349`

Build and tests: 26-27 October 2026

Cleanup completed: 27 October 2026, 19:51-19:55 IST

GitHub: https://github.com/rahulkmr1502/EWB-AWS-VPC-Peering

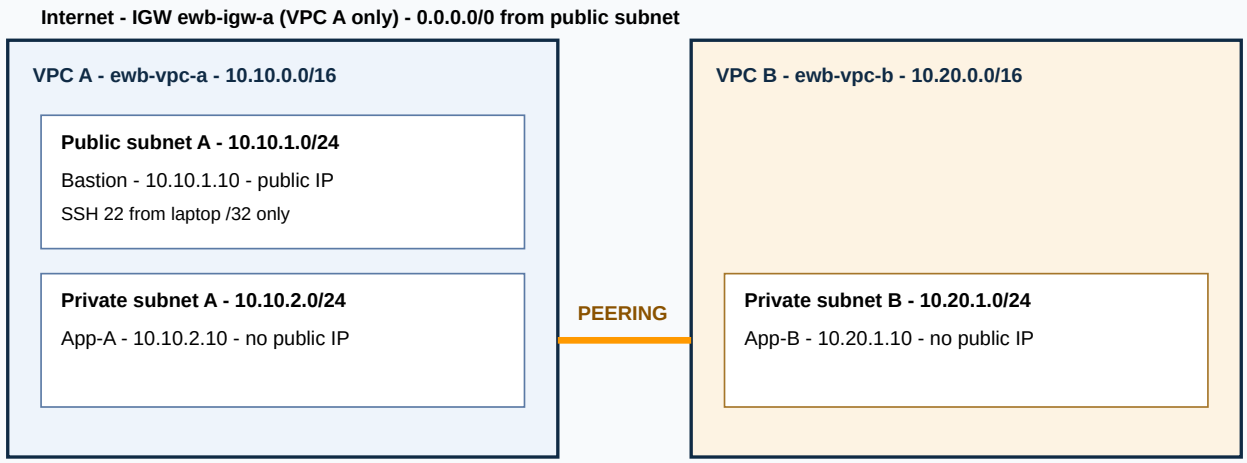
2 VPCs | 3 EC2 instances | 1 peering connection | Full cleanup verified

All resource IDs, IP addresses, test results and screenshots in this report are from the author's own build. No private key, password or AWS credential is included.

Project overview and architecture

This project builds two Amazon Virtual Private Clouds (VPCs) with non-overlapping address ranges and connects them with a VPC peering connection. VPC A holds a public subnet with a bastion host and a private subnet with an application instance. VPC B holds a second private application instance. Neither application instance has a public IP address.

Access follows one path: the author's laptop connects by Secure Shell (SSH) to the bastion, and from the bastion to the private instances using SSH agent forwarding, so the private key never leaves the laptop. Connectivity between the VPCs is tested with Internet Control Message Protocol (ICMP) ping.



Architecture: the bastion is the only instance with a public IP. App-A and App-B are private and are reached through the bastion. The peering connection carries only traffic between the two VPC CIDRs; it does not give VPC B internet access through VPC A.

IP address plan

Resource	Name	Resource ID	CIDR / IP
VPC A	ewb-vpc-a	`vpc-04ea62d1372d46a84`	`10.10.0.0/16`
VPC B	ewb-vpc-b	`vpc-00890b1d02b980903`	`10.20.0.0/16`
Public subnet A	ewb-a-public	`subnet-0cc314d68f2ad64a3`	`10.10.1.0/24`
Private subnet A	ewb-a-private	`subnet-00b4978fd24dc387c`	`10.10.2.0/24`
Private subnet B	ewb-b-private	`subnet-0f16a1a0cfddfd671`	`10.20.1.0/24`

All subnets are in availability zone ap-southeast-2a. The public subnet auto-assigns a public IPv4 address; the two private subnets do not. The non-overlapping CIDRs are a requirement for VPC peering.

Subnets, gateway, routes and instances

Internet gateway

Internet gateway **ewb-igw-a** (`igw-023b05519298e41a5`) is attached to VPC A only. No NAT gateway was used, so the private instances have no internet route. Only the public route table sends internet traffic to the gateway.

Route tables

Route table	ID	Routes
Public A - ewb-rt-a-public	`rtb-08c4c23e528ea6b6c`	`10.10.0.0/16 local; 0.0.0.0/0 to IGW; 10.20.0.0/16 to peering`
Private A - ewb-rt-a-private	`rtb-0bb077ef11ffcb0c2`	`10.10.0.0/16 local; 10.20.0.0/16 to peering`
Private B - ewb-rt-b-private	`rtb-073c364f56372b9bc`	`10.20.0.0/16 local; 10.10.0.0/16 to peering`

The peering routes shown here were added after the peering connection became Active. The main route tables are `rtb-0338c12bdf003083a` (VPC A) and `rtb-072bd1fc7a492f041` (VPC B) and were not used for the project subnets.

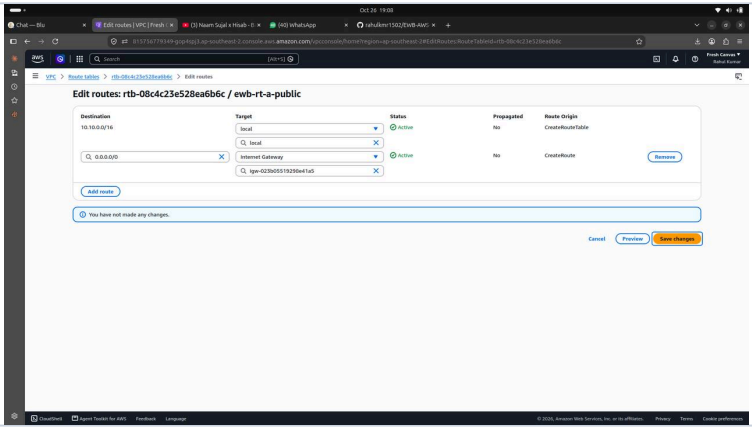


Figure 2 - Public route table A: local route and 0.0.0.0/0 route to the internet gateway.

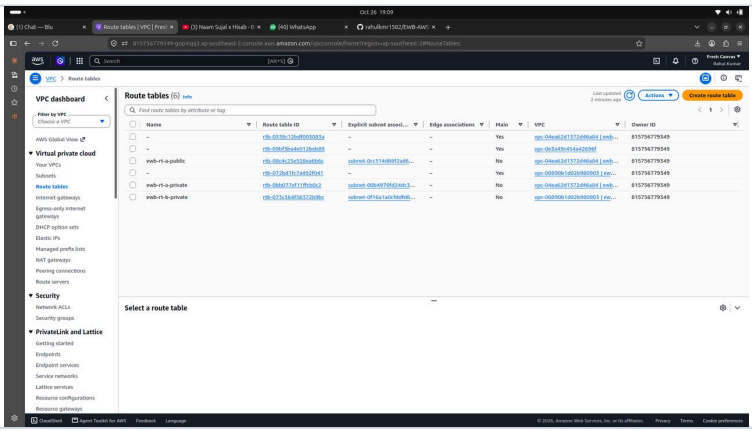


Figure 3 - Route table associations for the three project subnets.

EC2 instances ran private except the bastion

EC2 instances - Amazon Linux 2023, t3.micro, key pair ewb-lab

Instance	Instance ID	Private IP	Public IP
Bastion - ewb-bastion	`i-06861cf28c434c4d9`	`10.10.1.10`	`13.239.235.70 at test time`
App-A - ewb-app-a	`i-07a0bb1697e20011b`	`10.10.2.10`	None
App-B - ewb-app-b	`i-0614bec9b955b71b`	`10.20.1.10`	None

The bastion public IP changed on restart (earlier addresses were released on stop); its private IP, which the security groups rely on, stayed 10.10.1.10.

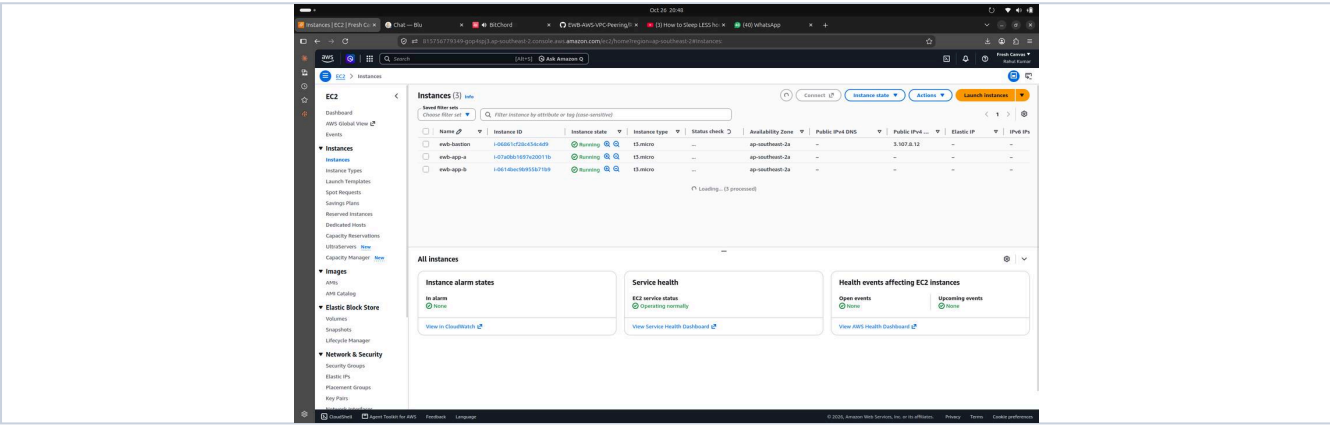


Figure 4 - All three instances Running before testing.

Peering needed routes in both directions

Peering connection **ewb-a-b-peer** (`pcx-0c41e6177ee14d14b`) connects VPC A as requester to VPC B as accepter in the same region and account. It was verified **Active** on 27 October 2026 at 18:55 IST.

An Active peering connection alone does not move traffic. Each side also needs a route to the other VPC's CIDR pointing at the peering connection, and the return route is just as necessary: without it, ping requests can leave but replies have no way back.

From	Destination	Target
Public route table A and Private route table A	`10.20.0.0/16`	`pcx-0c41e6177ee14d14b`
Private route table B	`10.10.0.0/16`	`pcx-0c41e6177ee14d14b`

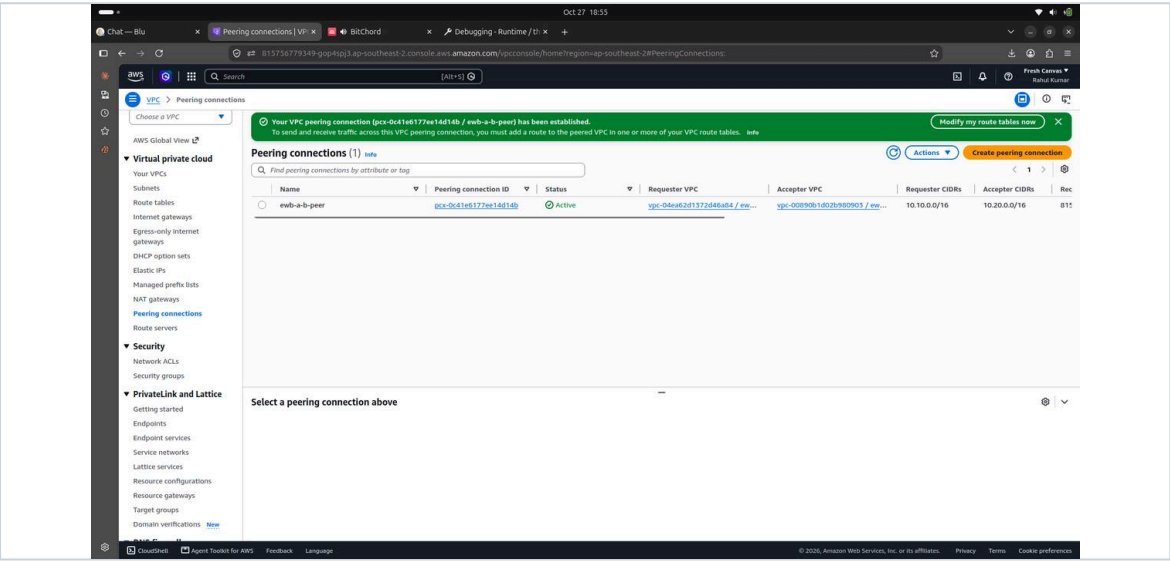


Figure 4 - Peering connection ewb-a-b-peer in Active state between VPC A (10.10.0.0/16) and VPC B (10.20.0.0/16).

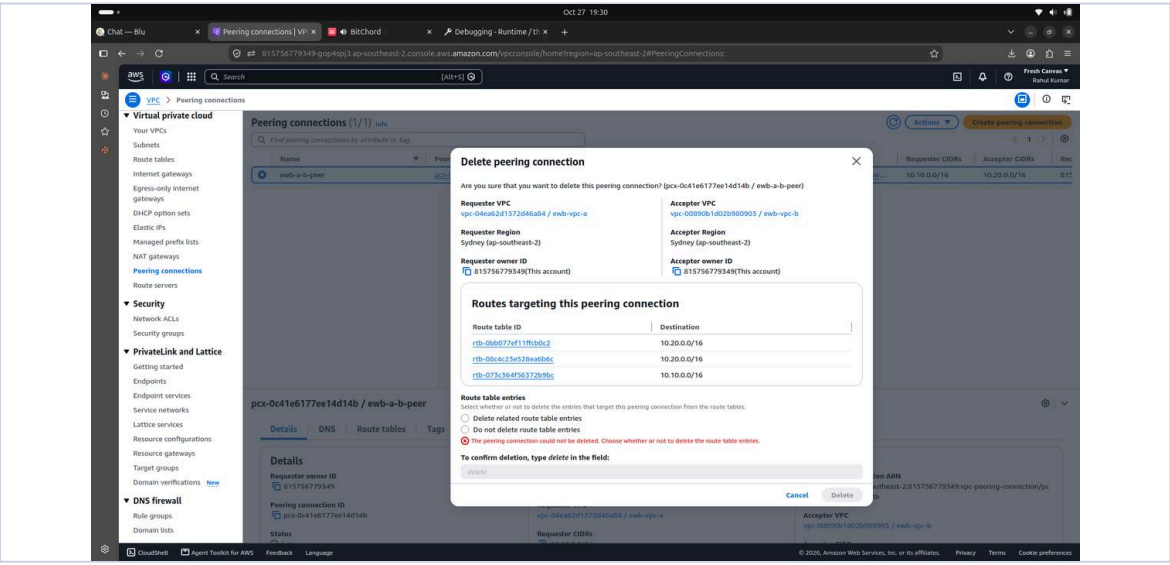


Figure 5 - The peering deletion dialog at cleanup lists all three peering routes that were in place, confirming routes existed in both directions.

VPC peering is non-transitive: VPC B cannot use VPC A's internet gateway through the peering connection, and a third VPC could not reach VPC B by passing through VPC A. Each pair of VPCs needs its own peering connection and its own routes.

Security groups allowed one source each

Every inbound rule allows exactly one source address with a /32 mask. SSH and ICMP were never opened to 0.0.0.0/0.

Security group	ID	Inbound rules
Bastion - ewb-sg-bastion	`sg-0907e0669898ab430`	SSH TCP 22 from the author's laptop only - `47.15.112.103/32` at test time
App-A - ewb-sg-app-a	`sg-0e089881b1795deb8`	SSH 22 from `10.10.1.10/32`; All ICMP-IPv4 from `10.20.1.10/32`
App-B - ewb-sg-app-b	`sg-08cfa1ce7771efb48`	SSH 22 from `10.10.1.10/32`; All ICMP-IPv4 from `10.10.2.10/32`

The bastion rule had to be updated on 27 October because the laptop's public IPv4 address had changed overnight, which blocked SSH until the rule was changed to the new /32 address. App-B sees the bastion's private IP, 10.10.1.10, as the SSH source, which is why both application security groups allow SSH from that single address.

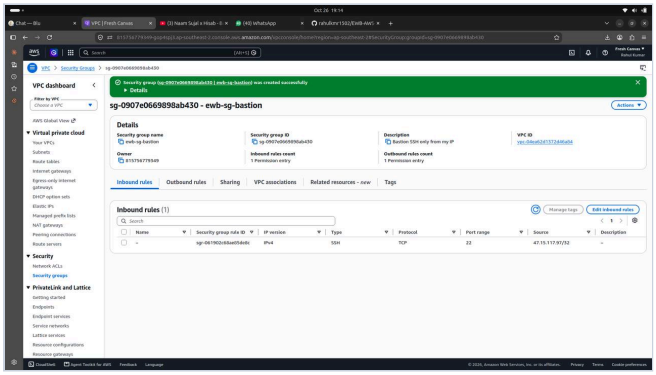


Figure 6 - Bastion security group: SSH from one /32 address only.

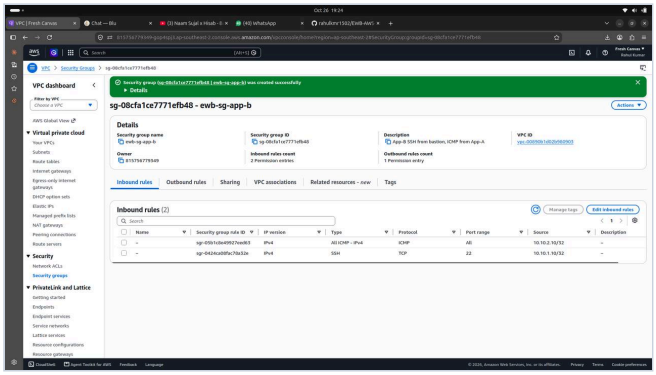


Figure 7 - App-B security group: SSH from the bastion and ICMP from App-A only.

Private SSH through the bastion

The laptop connects to the bastion with agent forwarding (`ssh -A -i ewb-lab.pem ec2-user@13.239.235.70`), then from the bastion to `10.10.2.10` or `10.20.1.10` with no key file on the bastion. The key stays on the laptop and the agent offers it for the second hop.

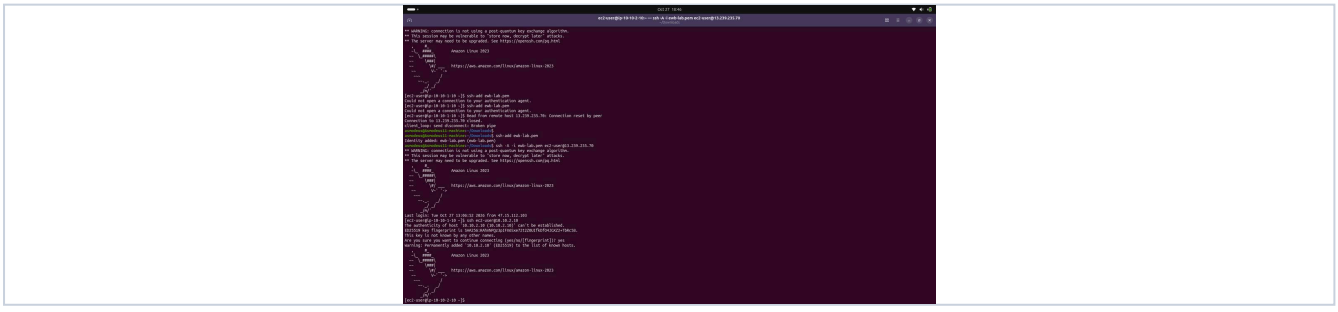


Figure 8 - SSH login to App-A through the bastion using agent forwarding.

Test evidence: ping failed before peering, then succeeded

Tests were run in a strict order. The failed ping had to be captured before the peering connection existed, because once peering and routes are in place that failure can no longer be reproduced honestly.

Test 1 - Before peering: ping fails

> **27 Oct 2026, 18:47 IST - App-A (10.10.2.10) to App-B (10.20.1.10): 4 packets transmitted, 0 received, 100% packet loss.** There was no route between the VPCs, so the ping could not reach App-B.

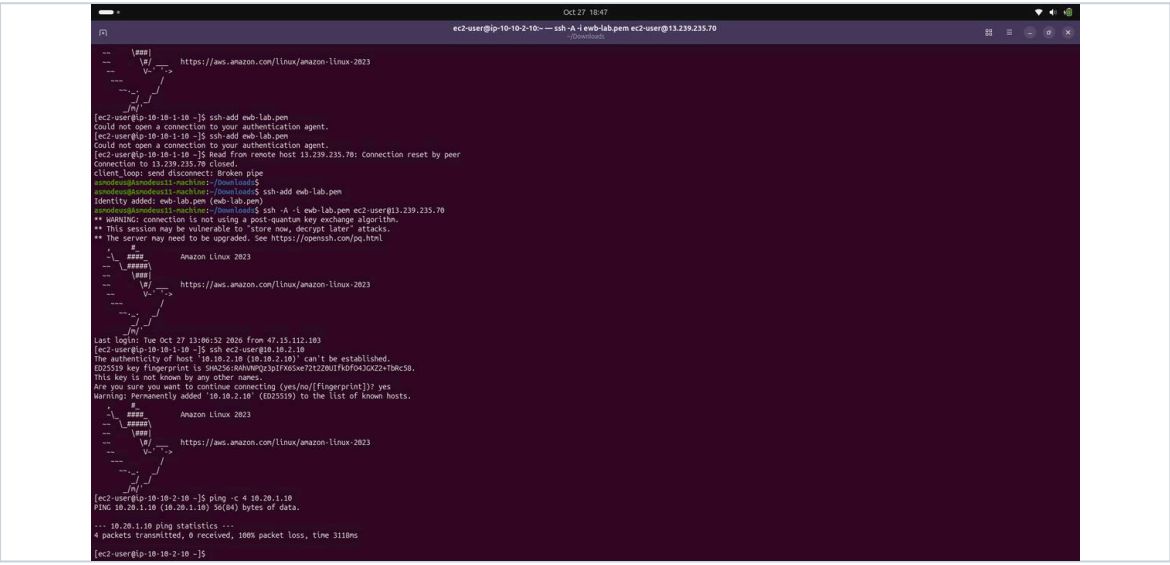


Figure 9 - Failed ping from App-A to App-B before peering: 100% packet loss.

Test 2 - After peering and routes: forward ping succeeds

> **27 Oct 2026, 19:16 IST - App-A to App-B: 4 transmitted, 4 received, 0% packet loss, round-trip average 0.291 ms.** All four ICMP replies returned from 10.20.1.10.

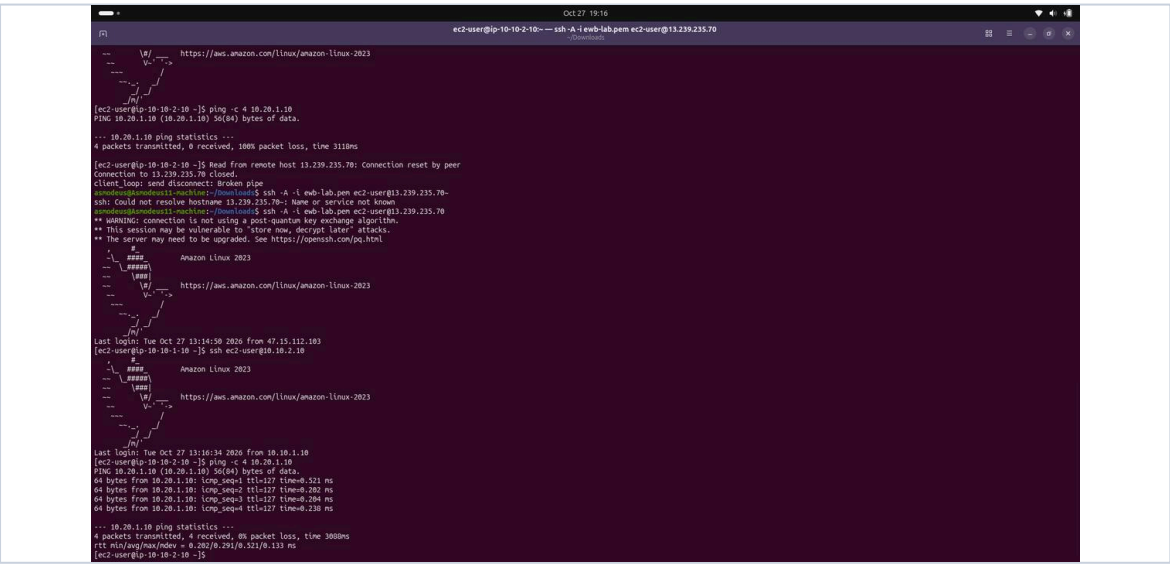


Figure 10 - Successful forward ping from App-A to App-B after peering routes were added: 0% packet loss.

Test evidence: reverse ping, SSH, and controlled failure

Test 3 - Reverse ping and SSH to App-B

> **27 Oct 2026, 19:18 IST - App-B (10.20.1.10) to App-A (10.10.2.10): 4 transmitted, 4 received, 0% packet loss, round-trip average 0.192 ms.** In the same session, SSH from the bastion reached App-B, shown by the prompt ec2-user@ip-10-20-1-10 - proof that a private instance in the other VPC is reachable through the bastion.

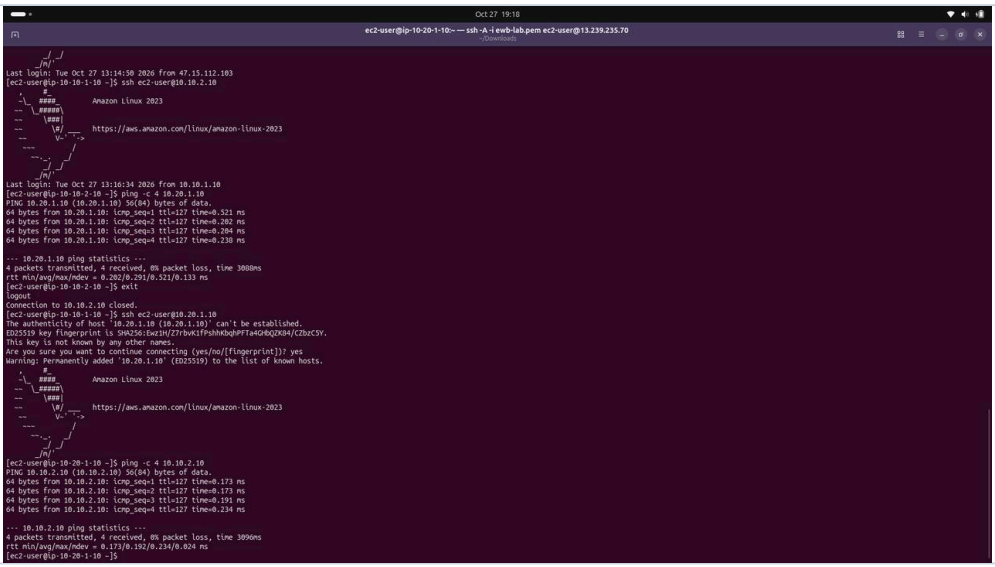


Figure 11 - SSH to App-B through the bastion and the successful reverse ping from App-B to App-A.

Controlled failure proved the security group controls the ping

Test 4 - Controlled failure: removing one rule kills the ping

> **27 Oct 2026, 19:22 IST - The All ICMP-IPv4 rule was removed from App-B's security group, leaving only SSH. The same App-A ping then failed again: 4 transmitted, 0 received, 100% packet loss.** Routing was unchanged, so this isolates the security group as the control point.

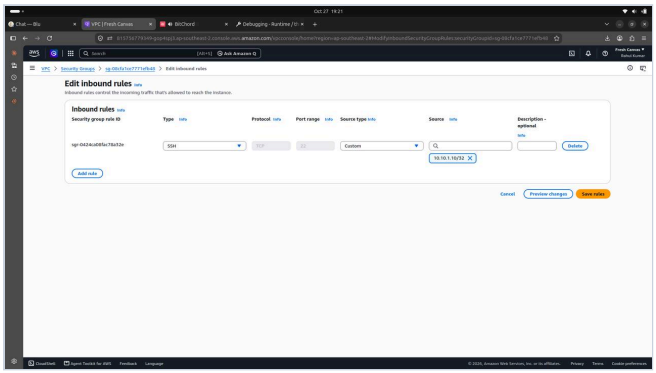


Figure 12 - App-B security group with the ICMP rule removed.

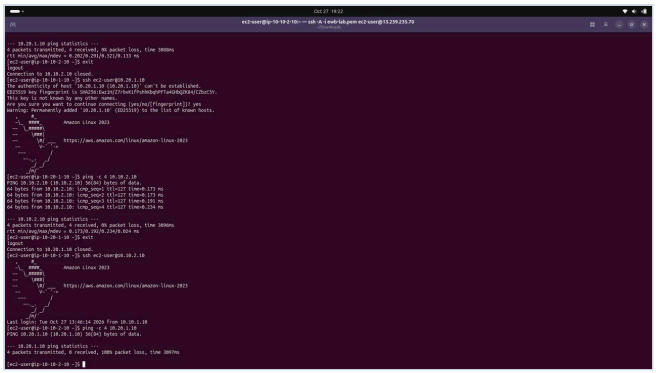


Figure 13 - Ping fails again after the ICMP rule is removed.

Test 5 - Recovery: restoring the rule revives the ping

> **27 Oct 2026, 19:24 IST - The ICMP rule was restored with source 10.10.2.10/32. The ping immediately succeeded again: 4 transmitted, 4 received, 0% packet loss, round-trip average 0.237 ms.**

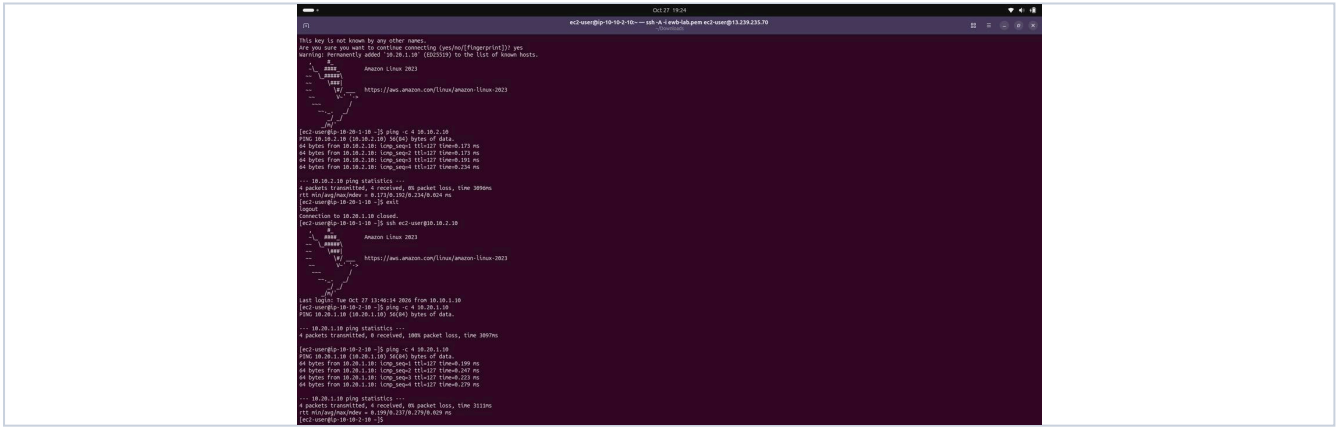


Figure 14 - Ping succeeds again after the ICMP rule is restored on App-B.

Cleanup removed every project resource

Cleanup followed the workbook's full cleanup order on 27 October 2026, 19:51-19:55 IST: terminate all EC2 instances, delete the peering connection, delete the custom route tables (after disassociating their subnets), delete the subnets, delete the security groups, detach and delete the internet gateway, then delete both VPCs.

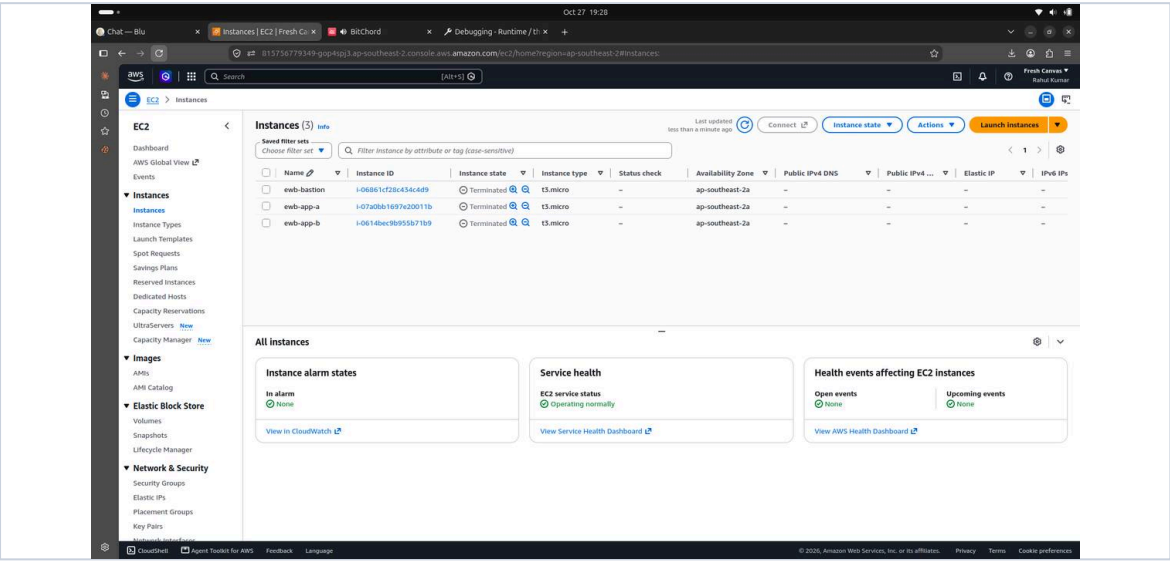


Figure 15 - All three instances - bastion, App-A and App-B - in Terminated state.

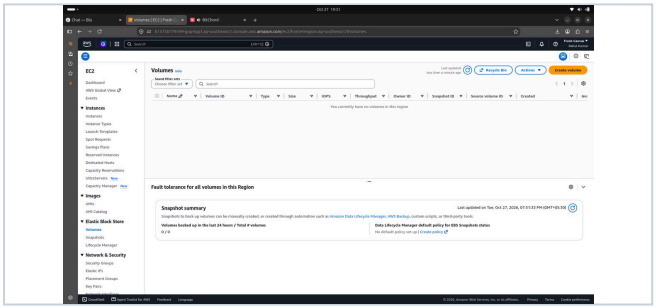


Figure 16 - No EBS volumes remain in the region.

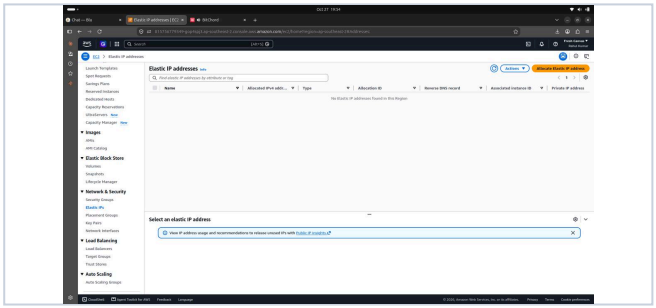


Figure 17 - No Elastic IP addresses remain in the region.

Cleanup proof: only the default VPC remains

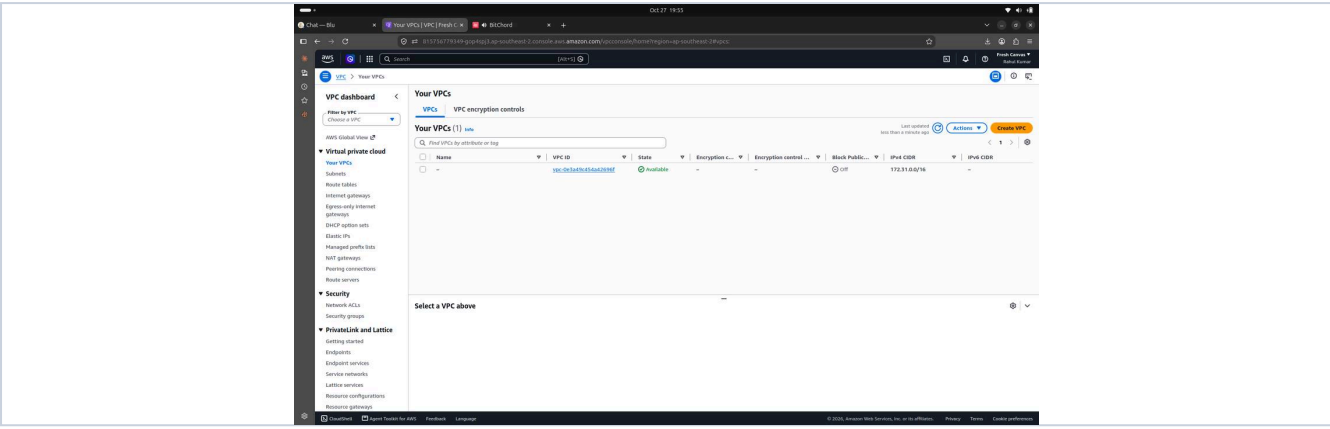


Figure 18 - Final VPC list: only the default VPC (vpc-0e3a49c454a42696f, 172.31.0.0/16) remains. Snapshots were also checked: none remain.

Conclusion

Two private VPCs with non-overlapping CIDRs were connected by a peering connection and proved end to end: a ping that failed before peering, successful pings in both directions after peering and routes were added, SSH to a private instance in the other VPC through the bastion, and a controlled failure and recovery driven by a single security-group rule. Every resource was then deleted, so nothing from this project is left running or billing.

Key lessons: an Active peering state is not proof of connectivity - routes in both directions and security-group rules decide whether traffic flows; the return route matters as much as the forward route; and least-privilege /32 rules work, but they must be updated when the source address changes.

Appendix: complete resource inventory

Resource	Name	ID
AWS account	Rahul Kumar	`815756779349`
Region	Asia Pacific (Sydney)	`ap-southeast-2`
VPC A	ewb-vpc-a	`vpc-04ea62d1372d46a84 - 10.10.0.0/16`
VPC B	ewb-vpc-b	`vpc-00890b1d02b980903 - 10.20.0.0/16`
Subnets	ewb-a-public / ewb-a-private / ewb-b-private	`subnet-0cc314d68f2ad64a3 / subnet-00b4978fd24dc387c / subnet-0f16a1a0cfddfd671`
Internet gateway	ewb-igw-a	`igw-023b05519298e41a5`
Route tables (custom)	ewb-rt-a-public / ewb-rt-a-private / ewb-rt-b-private	`rtb-08c4c23e528ea6b6c / rtb-0bb077ef11ffcb0c2 / rtb-073c364f56372b9bc`
Peering	ewb-a-b-peer	`pcx-0c41e6177ee14d14b`
Security groups	ewb-sg-bastion / ewb-sg-app-a / ewb-sg-app-b	`sg-0907e0669898ab430 / sg-0e089881b1795deb8 / sg-08cfa1ce7771efb48`
EC2	ewb-bastion / ewb-app-a / ewb-app-b	`i-06861cf28c434c4d9 / i-07a0bb1697e20011b / i-0614bec9b955b71b`
Key pair	ewb-lab (RSA)	`key-009dd9128638d8426`
Default VPC (retained)	-	`vpc-0e3a49c454a42696f - 172.31.0.0/16`

Test summary

Test	Time (IST, 27 Oct 2026)	Result
Ping before peering, App-A to App-B	18:47	100% packet loss - failed as expected
Peering Active	18:55	pcx-0c41e6177ee14d14b Active
Ping after peering, App-A to App-B	19:16	0% loss, avg 0.291 ms
Ping after peering, App-B to App-A, plus SSH to App-B	19:18	0% loss, avg 0.192 ms; SSH succeeded
Controlled failure, ICMP rule removed	19:22	100% packet loss
Recovery, ICMP rule restored	19:24	0% loss, avg 0.237 ms
Full cleanup	19:51-19:55	All project resources deleted; 0 volumes, 0 snapshots, 0 Elastic IPs

Screenshot evidence for every step above is in the project repository under `screenshots/` (files 01-31), at https://github.com/rahulkmr1502/EWB-AWS-VPC-Peering

Report prepared by Rahul Kumar for the EWB Courses AWS Internship. Amazon Web Services, AWS and Amazon VPC are trademarks of Amazon.com, Inc. or its affiliates.